



Escola Politécnica

NÚCLEO DE ELETIVAS INTERESCOLAS DA ESCOLA POLITÉCNICA

Curso: Ciências da Computação

Disciplina: Sistemas Operacionais

Professor: Felipe Viel

Avaliação M2 – Gerência de Memória

Acadêmicos:

Beatriz Pimentel Bagesteiro Alves - beatriz.8267731@edu.univali.br

Eduardo José de Souza - eduardo.8288917@edu.univali.br

Lucas Lopes Baroni - lucas.baroni@edu.univali.br

Itajaí, junho, 2026

Sumário

1. INTRODUÇÃO.....	3
2. METODOLOGIA.....	3
3. DESENVOLVIMENTO.....	4
3.1 ESTRUTURA.....	4
3.2 CONTEXTO DO PROBLEMA.....	4
3.3 SOLUÇÃO PROPOSTA.....	5
3.4 POLÍTICA FIRST FIT.....	6
3.5 FRAGMENTAÇÃO E COALESCÊNCIA.....	6
4. IMPLEMENTAÇÃO.....	6
4.1 INICIALIZAÇÃO.....	6
4.2 ALINHAMENTO DE MEMÓRIA.....	7
4.3 BUSCA POR BLOCO LIVRE.....	7
4.4 DIVISÃO DE BLOCOS.....	8
4.5 ALOCAÇÃO DINÂMICA COM KMALLOC().....	9
4.6 DESALOCAÇÃO COM KFREE().....	9
4.7 COALESCÊNCIA DE BLOCOS LIVRES.....	10
4.8 ESTATÍSTICAS DO HEAP.....	10
4.9 INTEGRAÇÃO COM TASKS.....	11
4.10 AJUSTES COMPLEMENTARES EM RELAÇÃO AO PROJETO ORIGINAL.....	12
5. TESTES E RESULTADOS OBTIDOS.....	12
5.1 CENÁRIOS DE TESTE.....	12
5.2 RESULTADOS DAS ESTATÍSTICAS.....	13
5.3 TESTE COM CRIAÇÃO E REMOÇÃO DE TASKS.....	13
5.4 EXEMPLO DE SAÍDA.....	14
6. ANÁLISE E DISCUSSÃO.....	14
7. REFERÊNCIAS BIBLIOGRÁFICAS.....	15

1. INTRODUÇÃO

Este relatório apresenta o desenvolvimento da Avaliação M2 da disciplina de Sistemas Operacionais, cujo objetivo foi evoluir o sistema de gerência de memória do projeto BasicMicrokernel.

O trabalho consistiu na substituição do alocador linear originalmente utilizado pelo microkernel por um alocador dinâmico baseado em Free List Allocator com política First Fit. A documentação disponibilizada pelo professor descreve a estrutura do projeto, o processo de build, o linker script, o boot, o driver UART, o scheduler, a criação de tasks e o allocator original.

Por se tratar de um sistema operacional minimalista, o gerenciamento de memória é implementado diretamente no código do kernel, sem uso de bibliotecas externas e sem dependência de malloc() da libc.

A proposta do trabalho está relacionada a conceitos centrais de sistemas operacionais, como heap, alocação dinâmica, desalocação, fragmentação, reutilização de memória, coalescência de blocos livres e integração entre o gerenciador de memória e a criação de tarefas. Esses elementos aproximam o microkernel de mecanismos presentes em sistemas operacionais reais, nos quais a memória precisa ser gerenciada de forma eficiente, segura e reutilizável durante a execução.

2. METODOLOGIA

A metodologia adotada foi baseada na análise do projeto BasicMicrokernel, na leitura da documentação técnica fornecida pelo professor e na identificação do funcionamento do allocator original.

Em seguida, foi feita a implementação progressiva de um novo módulo de memória. O desenvolvimento manteve a linguagem C, conforme exigido, e preservou a compatibilidade com a estrutura existente do kernel. As principais etapas foram:

- Estudo do módulo de memória original;
- Leitura da documentação do microkernel, incluindo build, linker script, boot, UART, heap, tasks e scheduler;
- Análise da organização do heap no projeto;
- Definição da estrutura de metadados para cada bloco de memória;
- Implementação da inicialização do heap;
- Implementação da função kmalloc() com política First Fit;

- Implementação da função `kfree()`;
- Implementação da divisão de blocos livres;
- Implementação da coalescência de blocos adjacentes;
- Implementação das estatísticas do heap;
- Integração da alocação dinâmica com a criação de tasks;
- Criação de cenários de teste para validar alocação, liberação, reutilização e fragmentação.

A validação foi feita por inspeção da implementação e pela simulação dos cenários definidos no arquivo de testes do projeto, considerando o comportamento do heap e os valores de uso e disponibilidade de memória produzidos pelo algoritmo.

3. DESENVOLVIMENTO

3.1 ESTRUTURA

O processo de boot é iniciado em Assembly, configura registradores essenciais da arquitetura RISC-V, inicializa a stack do kernel, limpa a seção `.bss` e chama `kernel_main()`. A comunicação textual com o ambiente de execução é feita pelo driver UART, mapeado em memória, o que permite imprimir mensagens no QEMU sem depender de `printf()` ou de biblioteca padrão.

O sistema de build utiliza um Makefile com toolchain cruzada `riscv64-unknown-elf`, flags de ambiente `freestanding` e um linker script próprio. O linker script define o ponto de entrada, organiza as seções `.text`, `.rodata`, `.data` e `.bss` e garante que o binário `kernel.elf` seja posicionado corretamente para execução no ambiente RISC-V.

No nível de execução, o microkernel possui um mecanismo simples de tasks, com TCBs contendo registradores, ponteiro de entrada, prioridade e stack. O scheduler utiliza uma política Round-Robin e a troca de contexto é feita por rotina em Assembly. A nova gerência de memória precisa, portanto, encaixar-se nesse fluxo: o heap deve ser inicializado antes da criação das tasks, e as stacks devem ser obtidas dinamicamente pelo allocator do kernel.

3.2 CONTEXTO DO PROBLEMA

O microkernel original utilizava um alocador linear, também conhecido como Bump Allocator. No código-base, o heap era definido a partir do endereço fixo `HEAP_START 0x80400000UL`, com tamanho de 8 MB, e controlado por três ponteiros: `heap_base`, `heap_end` e

heap_ptr. Nesse modelo, cada nova alocação apenas avança heap_ptr dentro da área do heap. A simplicidade do método facilita a implementação inicial, mas cria limitações importantes em um sistema que precisa manter a execução contínua.

As principais limitações do alocador linear no projeto original são:

- Não permite reutilização de memória previamente alocada;
- kfree() existe apenas como função mínima, mas não devolve memória ao heap;
- memory_used() e memory_free() dependem apenas da diferença entre heap_ptr, heap_base e heap_end;
- Provoca exaustão progressiva do heap;
- Não trata fragmentação;
- Limita o gerenciamento dinâmico de tasks.

Dessa forma, quando uma área de memória deixa de ser necessária, ela não retorna ao conjunto de memória disponível. Em um kernel, esse comportamento é problemático, pois a criação e destruição de estruturas internas, como stacks de tasks, pode consumir progressivamente o heap até a indisponibilidade de memória.

3.3 SOLUÇÃO PROPOSTA

A solução implementada substitui o modelo linear por um Free List Allocator. No código final, o heap passou a ser representado por um vetor estático interno de 65.536 bytes, definido por HEAP_SIZE 0x10000. Esse tamanho foi adotado na implementação entregue e é usado nos testes e estatísticas apresentados neste relatório. Nesse novo modelo, o heap é dividido em blocos, e cada bloco possui um cabeçalho com metadados. Esses metadados informam o tamanho do bloco, se ele está livre ou ocupado e qual é o próximo bloco da lista. A estrutura utilizada no projeto foi:

```
typedef struct block {  
    uint64_t size;  
    int free;  
    struct block *next;  
} block_t;
```

Figura 01 - Implementação da struct block.

Fonte: elaborado pelo autor.

Com essa abordagem, o heap passa a ser representado por uma lista encadeada de blocos. A alocação percorre a lista em busca do primeiro bloco livre que possua espaço suficiente para atender à requisição. Essa política é chamada de First Fit.

3.4 POLÍTICA FIRST FIT

A política First Fit percorre a lista de blocos desde o início e seleciona o primeiro bloco livre com tamanho suficiente para a alocação solicitada. Essa política possui baixo custo de implementação e tende a ser eficiente para um microkernel educacional, pois evita a necessidade de buscar o melhor bloco possível em toda a lista. A principal vantagem é a simplicidade e a rapidez média de alocação.

A principal desvantagem é a possibilidade de fragmentação, pois blocos pequenos podem permanecer espalhados no heap. Para reduzir esse problema, a implementação utiliza divisão de blocos e coalescência.

3.5 FRAGMENTAÇÃO E COALESCÊNCIA

A fragmentação ocorre quando o heap possui memória livre, mas dividida em pequenos blocos que podem não atender a uma requisição maior. Para minimizar esse problema, a implementação realiza coalescência, isto é, a união de blocos livres adjacentes.

Quando `kfree()` libera um bloco, ele é marcado como livre e, em seguida, o algoritmo percorre a lista verificando se existem blocos livres consecutivos. Quando dois blocos adjacentes estão livres, eles são unidos em um único bloco maior, somando também o espaço ocupado pelo cabeçalho do bloco absorvido.

4. IMPLEMENTAÇÃO

4.1 INICIALIZAÇÃO

A função `memory_init()` inicializa o heap com um único bloco livre. Esse bloco começa no endereço base do vetor heap e ocupa todo o espaço disponível, descontado o tamanho do cabeçalho.

```
void memory_init(void) {
    free_list = (block_t*) heap;
    free_list->size = HEAP_SIZE - sizeof(block_t);
    free_list->free = 1;
    free_list->next = 0;
}
```

Figura 02 - Implementação da função *memory_init()*.

Fonte: elaborado pelo autor.

Esse procedimento atende ao requisito de preparar o heap para a alocação dinâmica, configurando a lista livre inicial.

4.2 ALINHAMENTO DE MEMÓRIA

A implementação adiciona alinhamento automático em múltiplos de 8 bytes. Esse recurso é importante em arquitetura de 64 bits, pois mantém os endereços e tamanhos compatíveis com acessos alinhados.

```
static uint64_t align_up(uint64_t s) {
    return (s + ALIGN - 1) & ~(uint64_t)(ALIGN - 1);
}
```

Figura 03 - Implementação da função *align_up()*.

Fonte: elaborado pelo autor.

Embora o alinhamento automático tenha sido listado como funcionalidade extra opcional, ele foi implementado no projeto.

4.3 BUSCA POR BLOCO LIVRE

A busca por memória disponível é feita pela função *pesquisar()*, que percorre a lista encadeada até encontrar o primeiro bloco livre com tamanho suficiente.

```
static block_t *pesquisar(uint64_t r_size) {
    block_t *aux = free_list;
    while (aux != NULL) {
        if (aux->free == 1 && aux->size >= r_size) {
            return aux;
        }
        aux = aux->next;
    }
    return NULL;
}
```

Figura 04 - Implementação da função *pesquisar()*.

Fonte: elaborado pelo autor.

Esse trecho representa diretamente a política First Fit, pois o primeiro bloco compatível é selecionado.

4.4 DIVISÃO DE BLOCOS

Quando um bloco livre é maior que o tamanho solicitado, ele pode ser dividido em dois: um bloco ocupado, com o tamanho requisitado, e um novo bloco livre com o espaço restante.

```
static void dividir_bloco(block_t *bloco, uint64_t size) {
    uint64_t espaco_restante = bloco->size - size;

    if (espaco_restante <= BLOCK_HEADER_SIZE + ALIGN)
        return;

    block_t *novo = (block_t *)((uint8_t *)bloco + BLOCK_HEADER_SIZE + size);

    novo->size = espaco_restante - BLOCK_HEADER_SIZE;
    novo->free = 1;
    novo->next = bloco->next;

    bloco->size = size;
    bloco->next = novo;
}
```

Figura 05 - Implementação da função *dividir_bloco()*.

Fonte: elaborado pelo autor.

A divisão evita desperdício de memória, pois impede que uma pequena alocação ocupe um bloco livre muito maior do que o necessário.

4.5 ALOCAÇÃO DINÂMICA COM KMALLOC()

A função `kmalloc()` valida o tamanho solicitado, aplica alinhamento, procura um bloco livre com First Fit, divide o bloco quando possível e retorna o ponteiro para a área útil, localizada logo após o cabeçalho.

```
void *kmalloc(uint64_t size) {
    if (size == 0)
        return NULL;

    size = align_up(size);

    block_t *bloco = pesquisar(size);
    if (bloco == NULL)
        return NULL;

    dividir_bloco(bloco, size);
    bloco->free = 0;

    return (void *)((uint8_t *)bloco + BLOCK_HEADER_SIZE);
}
```

Figura 06 - Implementação da função *kmalloc()*.

Fonte: elaborado pelo autor.

A função não utiliza `malloc()` da `libc` e opera exclusivamente sobre o heap interno do kernel.

4.6 DESALOCAÇÃO COM KFREE()

A função `kfree()` recebe um ponteiro para a área útil do bloco, recupera o cabeçalho subtraindo o tamanho do metadado, marca o bloco como livre e chama a coalescência.

```
void kfree(void *ptr) {
    if (ptr == NULL)
        return;

    block_t *bloco = (block_t *)((uint8_t *)ptr - BLOCK_HEADER_SIZE);
    bloco->free = 1;

    coalesce();
}
```

Figura 07 - Implementação da função *kfree()*.

Fonte: elaborado pelo autor.

Com isso, a memória liberada pode ser reutilizada por chamadas futuras de `kmalloc()`.

4.7 COALESCÊNCIA DE BLOCOS LIVRES

A coalescência percorre a lista e une blocos livres consecutivos. Isso reduz a fragmentação externa e aumenta a chance de atender futuras alocações maiores.

```
static void coalesce(void) {
    block_t *atual = free_list;
    while (atual != NULL && atual->next != NULL) {
        if (atual->free == 1 && atual->next->free == 1) {
            atual->size += BLOCK_HEADER_SIZE + atual->next->size;
            atual->next = atual->next->next;
        } else {
            atual = atual->next;
        }
    }
}
```

Figura 08 - Implementação da função *coalesce()*.

Fonte: elaborado pelo autor.

4.8 ESTATÍSTICAS DO HEAP

Foram implementadas as funções obrigatórias *memory_total()*, *memory_used()* e *memory_free()*. Elas permitem acompanhar o estado do heap durante a execução dos testes.

```
uint64_t memory_total(void) {
    return (uint64_t)HEAP_SIZE;
}

uint64_t memory_used(void) {
    uint64_t used = 0;
    block_t *aux = free_list;
    while (aux != NULL) {
        if (aux->free == 0)
            used += BLOCK_HEADER_SIZE + aux->size;
        aux = aux->next;
    }
    return used;
}

uint64_t memory_free(void) {
    uint64_t free = 0;
    block_t *aux = free_list;
    while (aux != NULL) {
        if (aux->free == 1)
            free += aux->size;
        aux = aux->next;
    }
    return free;
}
```

Figura 10 - Implementação das funções *memory_total()*, *memory_used()*, *memory_free()*.

Fonte: elaborado pelo autor.

Também foi implementada a função *mapa_heap()*, que exibe o mapa de blocos do heap, com endereço, tamanho e status de cada bloco.

4.9 INTEGRAÇÃO COM TASKS

No projeto original, *xTaskCreate()* já chamava *kmalloc()* para reservar a stack da task, porém essa reserva ainda dependia do comportamento do Bump Allocator, sem liberação real posterior. Com o novo allocator, a mesma integração passa a ter reutilização efetiva de memória, pois a stack é alocada em blocos gerenciados pela lista livre.

```
t->stack = (uint8_t*)kmalloc(stack_size);
uint64_t *sp = (uint64_t*)(t->stack + stack_size);
```

Figura 11 - Implementação da nova abordagem orientada a blocos.

Fonte: elaborado pelo autor.

Além disso, foi implementada a função *vTaskDelete()*, que libera a stack da task removida por meio de *kfree()*.

```
void vTaskDelete(void (*task)(void)) {
    for (int i = 0; i < task_count; i++) {
        if (tasks[i].entry == task) {
            kfree(tasks[i].stack);
            tasks[i].stack = NULL;
            tasks[i] = tasks[task_count - 1];
            task_count--;
            return;
        }
    }
}
```

Figura 12 - Implementação da função *vTaskDelete()*.

Fonte: elaborado pelo autor.

Esse ponto atende ao requisito de integração do gerenciamento de memória com as tasks do microkernel.

4.10 AJUSTES COMPLEMENTARES EM RELAÇÃO AO PROJETO ORIGINAL

Além da troca do allocator, a comparação entre o código original e o código final mostra os seguintes ajustes:

- `memory.h` passou a declarar `memory_total()` e `mapa_heap()`;
- `task.h` passou a declarar `vTaskDelete()`;
- Foi criado o módulo `testes.c/testes.h` para concentrar os cenários de teste de memória;
- O `Makefile` passou a compilar `testes.o` e recebeu alvos auxiliares para limpar e executar o kernel no QEMU;
- `main.c` passou a chamar `testes_memoria()`, imprimir estatísticas, criar tasks, remover a Task 1 e exibir o mapa do heap;
- `context.S` foi ajustado para preservar o ponteiro do contexto novo em `t6` durante o carregamento dos registradores, evitando que o uso de `a1` como base fosse prejudicado quando o próprio registrador `a1` fosse restaurado;
- `string.c` recebeu implementação de `memcpy()`, embora essa alteração não seja o foco principal da gerência de memória.

Esses ajustes dão suporte aos testes e à demonstração do allocator implementado.

5. TESTES E RESULTADOS OBTIDOS

Os testes implementados no arquivo `testes.c` exercitam os principais requisitos da avaliação: múltiplas alocações, liberação, reutilização de blocos, divisão, coalescência e estatísticas do heap.

O heap possui tamanho total de 65.536 bytes. Considerando o cabeçalho de 24 bytes usado pela estrutura `block_t`, o primeiro bloco livre inicial possui 65.512 bytes úteis.

5.1 CENÁRIOS DE TESTE

Teste	Ação executada	Objetivo
1	Aloca ptr1 = 1000, ptr2 = 500 e ptr3 = 2000 bytes	Validar múltiplas alocações consecutivas e divisão do bloco livre inicial.
2	Libera ptr2	Criar um bloco livre intermediário, simulando fragmentação externa.
3	Aloca ptr4 = 1000 bytes	Demonstrar a política First Fit ignorando o bloco livre insuficiente e usando o próximo bloco com espaço adequado.
4	Aloca ptr5 = 500 bytes	Demonstrar reutilização do bloco liberado anteriormente por kfree().
5	Libera ptr1 e ptr5	Demonstrar coalescência parcial, unindo blocos livres adjacentes.
6	Libera ptr3 e ptr4	Demonstrar coalescência final, retornando o heap a um único bloco livre.

Figura 13 - Listagem dos testes e seus objetivos.

Fonte: elaborado pelo autor.

5.2 RESULTADOS DAS ESTATÍSTICAS

Etapa	Blocos principais	Heap usado	Heap livre
Inicialização	1 bloco livre de 65.512 byte	0	65.512
Após ptr1, ptr2 e ptr3	3 blocos ocupados e 1 livre	3.576	61.936
Após liberar ptr2	2 blocos ocupados e 2 livres	3.048	62.440
Após alocar ptr4	3 blocos ocupados e 2 livres	4.072	61.416
Após alocar ptr5	4 blocos ocupados e 1 livre	4.600	60.912
Após liberar ptr1 e ptr5	2 blocos ocupados e 2 livres	3.048	62.440
Após liberar ptr3 e ptr4	1 bloco livre total	0	65.512

Figura 14 - Listagem dos resultados.

Fonte: elaborado pelo autor.

Os valores demonstram que a memória é corretamente reutilizada e que a coalescência consegue reconstruir um bloco livre total ao final das liberações.

5.3 TESTE COM CRIAÇÃO E REMOÇÃO DE TASKS

Após os testes de alocação direta, o kernel cria duas tasks com stacks de 2048 bytes. Cada stack é alocada dinamicamente pelo novo kmalloc().

Etapa	Descrição	Heap usado	Heap livre
Criação da Task 1 e da Task 2	Duas stacks de 2048 bytes são alocadas dinamicamente com <code>kmalloc()</code> .	4.144 bytes	61.368 bytes
Remoção da Task 1	A stack da Task 1 é liberada por <code>vTaskDelete()</code> usando <code>kfree()</code> .	2.072 bytes	63.416 bytes

Figura 15 - Listagem de cada uma das etapas e suas estatísticas.

Fonte: elaborado pelo autor.

Esse resultado demonstra que o gerenciamento de memória foi integrado ao sistema de tasks e que a memória de stacks pode retornar ao heap após a remoção da task.

5.4 EXEMPLO DE SAÍDA

```

Heap Total: 65536
Heap Usado: 4144
Heap Livre: 61368
Task 1 criada
Task 2 criada
Task 1 deletada
Heap Total: 65536
Heap Usado: 2072
Heap Livre: 63416

```

Figura 16 - Exemplo de saída do algoritmo.

Fonte: elaborado pelo autor.

6. ANÁLISE E DISCUSSÃO

A implementação atende ao objetivo principal do trabalho, pois substitui o alocador linear por um Free List Allocator funcional com política First Fit. O novo modelo permite que a memória liberada retorne ao heap e seja reutilizada, eliminando a principal limitação do Bump Allocator.

A divisão de blocos é um ponto importante da solução, pois permite aproveitar blocos grandes sem desperdiçar todo o espaço em uma única alocação pequena. Ao mesmo tempo, a coalescência reduz os efeitos da fragmentação externa, unindo blocos livres adjacentes e aumentando a capacidade de atender futuras requisições maiores.

A política First Fit mostrou-se adequada para o contexto do microkernel. Ela é simples, possui baixo custo de processamento e facilita a compreensão do comportamento do allocator. Em comparação com Best Fit, o First Fit tende a ser mais rápido, pois não precisa percorrer

necessariamente toda a lista para encontrar o menor bloco possível. Em comparação com Worst Fit, evita a estratégia de sempre consumir os maiores blocos livres. Para um projeto educacional, a escolha equilibra simplicidade, desempenho e clareza.

A integração com tasks também é relevante. No modelo original, `xTaskCreate()` já chamava `kmalloc()` para reservar stack, mas essa alocação ocorria sobre um Bump Allocator cuja `kfree()` não reutilizava memória. Com a nova implementação, `xTaskCreate()` continua utilizando `kmalloc()`, agora sobre blocos livres encadeados, e `vTaskDelete()` utiliza `kfree()` para liberar a stack. Isso aproxima o microkernel de um comportamento mais realista, no qual recursos associados a tarefas podem ser devolvidos ao sistema.

O trabalho implementou um sistema de gerência dinâmica de memória para o BasicMicrokernel, utilizando Free List Allocator com política First Fit. A solução contempla inicialização do heap, alocação dinâmica, desalocação, divisão de blocos, coalescência, estatísticas de uso e integração com o sistema de tasks.

Os testes demonstram múltiplas alocações, liberação de memória, reutilização de blocos, tratamento de fragmentação e recuperação do heap por meio da coalescência. Assim, a implementação supera as limitações do alocador linear original e fornece uma base mais adequada para o gerenciamento dinâmico de memória no microkernel.

7. REFERÊNCIAS BIBLIOGRÁFICAS

VIEL, Felipe. BasicMicrokernel. Repositório de referência da disciplina de Sistemas Operacionais. Disponível em: <https://github.com/VielF/BasicMicrokernel>. Acesso em: 08 jun. 2026.

VIEL, Felipe. Um Microkernel Simples para RISC-V: Implementação de um Sistema Operacional Minimalista para disciplina de SO. Universidade do Vale do Itajaí, 2026.

TANENBAUM, Andrew S.; BOS, Herbert. Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Fundamentos de Sistemas Operacionais. Rio de Janeiro: LTC, 2015.